جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

Introduction to Artificial Intelligence (SE 444)

Lecture 7.2: Propositional Logic

Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture introduces the **fundamentals of propositional logic** as the foundation for knowledge-based agents in AI. You'll learn how to formally represent knowledge using logical symbols, understand truth conditions through semantics, and apply inference rules for automated reasoning. These concepts are essential for building intelligent systems that can reason logically about their environment.



🤖 Why Logic Matters for Intelligent Agents

Foundation of Intelligence

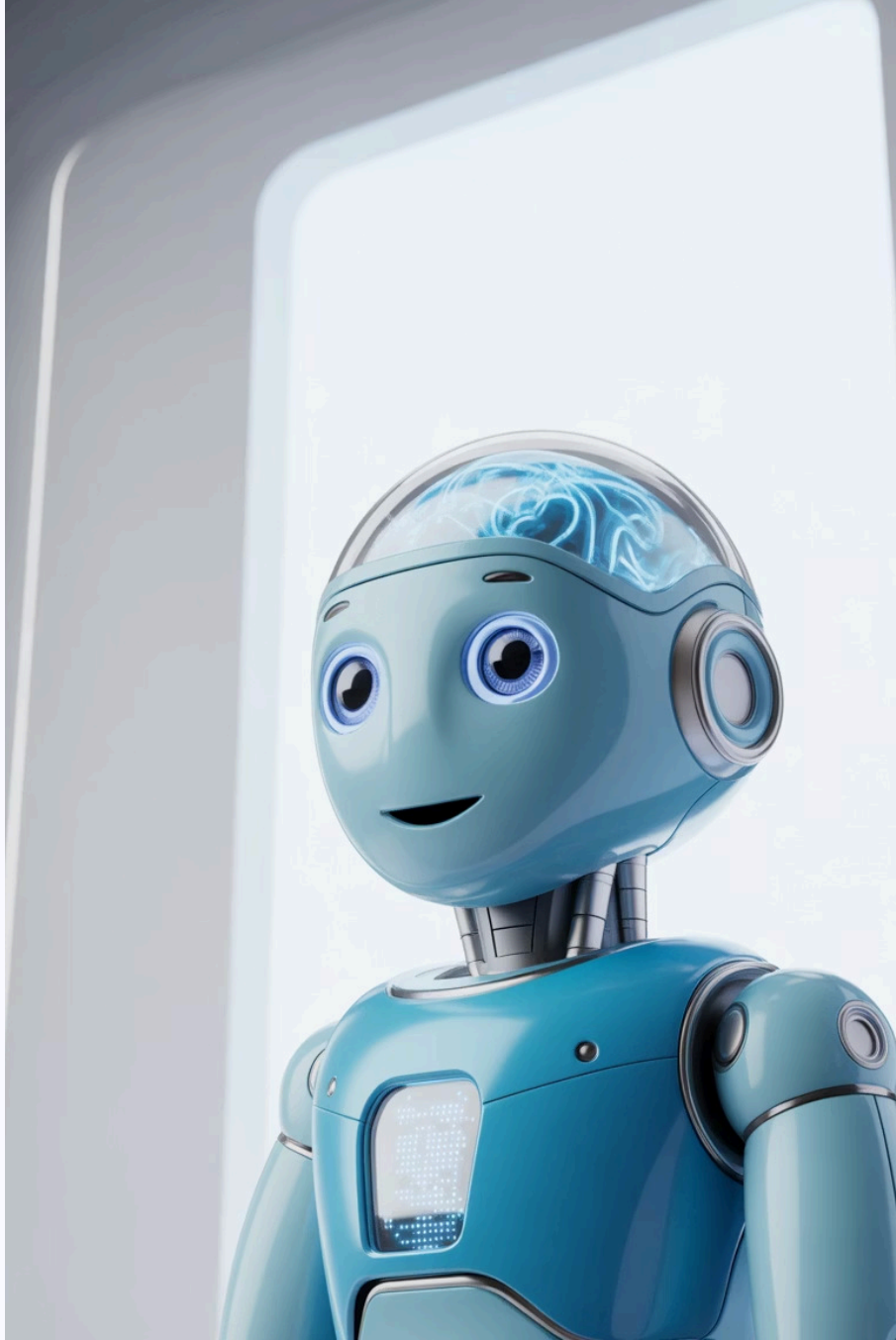
Knowledge-Based Agents rely on logic to represent and reason about the world systematically

Formal Language

Logic provides a formal language of truth and reasoning for AI systems

Evolution of Agency

Transforms agents from **"reactive"** systems → **"rational and explainable"** decision-makers





🔗 From Knowledge-Based Agents to Formal Logic

KBA Architecture Recap

Knowledge-Based Agents operate through a continuous cycle:

- **Tell:** Add new percepts to knowledge base
- **Ask:** Query what actions are appropriate
- **Act:** Execute chosen action in environment

The Missing Link 🔑

To implement this cycle, we need two critical components:

1. A **formal language** (syntax rules)
2. An **interpretation system** (semantics)

Enter: **Propositional Logic** 💡

📘 Definition of a Formal Language

A **formal system** provides the mathematical foundation for automated reasoning. It consists of two essential components that work together:

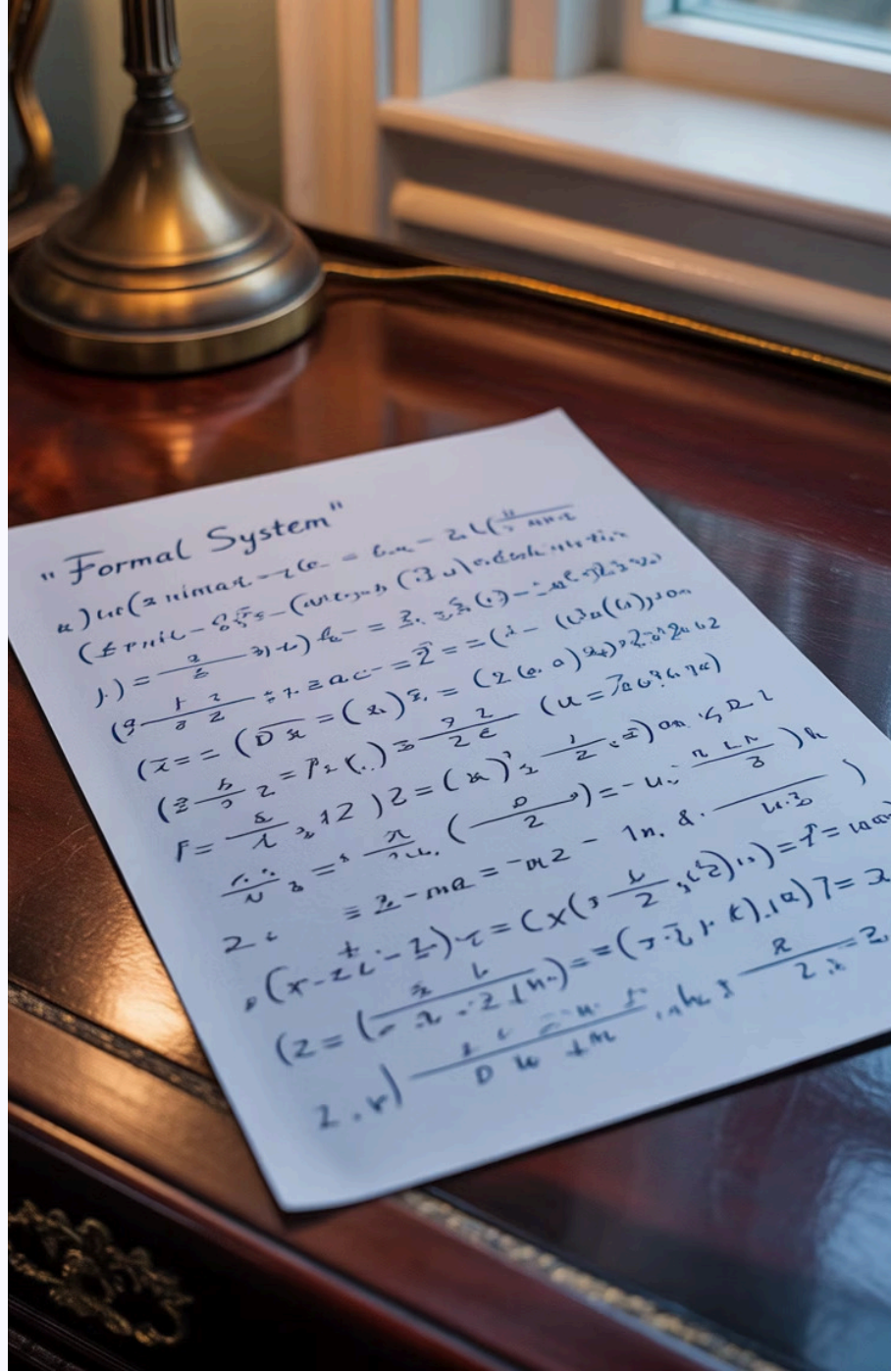
Syntax

Defines **how sentences are built** from basic symbols using grammatical rules and structural constraints

Semantics

Determines **what sentences mean** by assigning truth values and interpretations to symbolic expressions

Essential properties: Must be unambiguous, consistent, and interpretable by machines 🖥️



🧩 Propositional Logic Vocabulary

The building blocks of logical expressions consist of four fundamental categories:



Constants

True and **False** represent the fundamental logical truth values in every proposition



Propositions

P, Q, R denote atomic facts or basic statements about the world



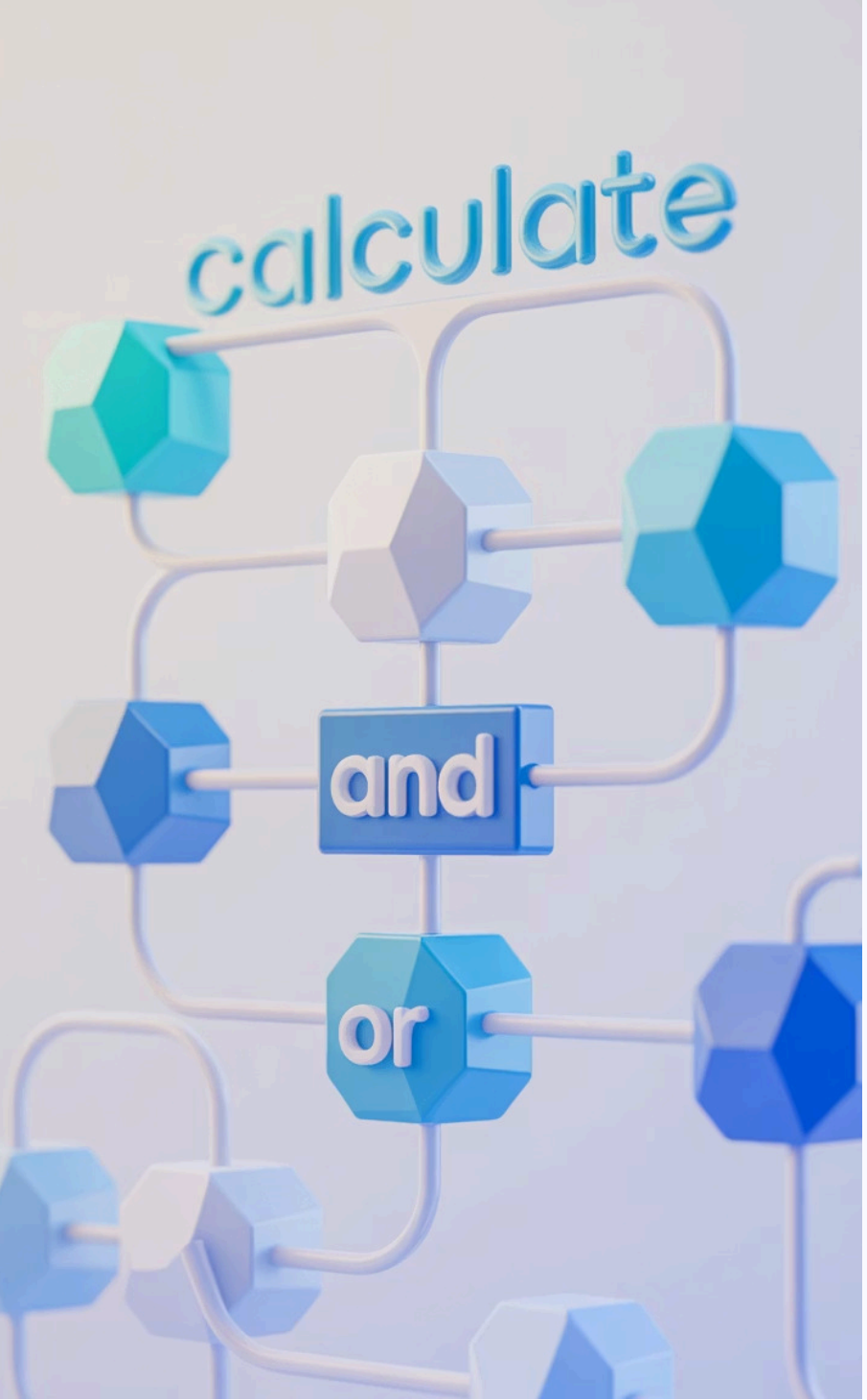
Connectives

¬, ∧, ∨, →, ↔️ are logical operators that combine propositions into complex expressions



Parentheses

( ) define structure and precedence, ensuring unambiguous interpretation



⚙️ Syntax Rules

01

Atomic Sentences

Single propositional symbols like **P**, **Q**, **R** are the simplest valid sentences

02

Negation Formation

If **α** is a sentence, then **¬α** (not α) is also a sentence

03

Binary Connectives

If **α** and **β** are sentences, we can form: **(α ∧ β)**, **(α ∨ β)**, **(α → β)**, **(α ↔ β)**

04

Well-Formed Formulas

A **WFF** is any validly structured sentence following these recursive rules

🧠 Examples of Syntax Validity

Understanding which logical expressions are properly formed is crucial for automated reasoning:

Sentence	Valid?	Reason
P ∧ Q	✅	Both are valid propositions with proper connective
(¬P ∨ (Q → R))	✅	Properly formed with correct nesting and operators
((P ∧ Q) ↔ R)	✅	Biconditional with valid subformula structure
¬ ∧ Q	❌	Missing operand for negation operator
P Q ∨	❌	Incorrect operator placement (postfix notation)
(P →)	❌	Implication requires two operands

💬 Abstraction in Propositional Logic



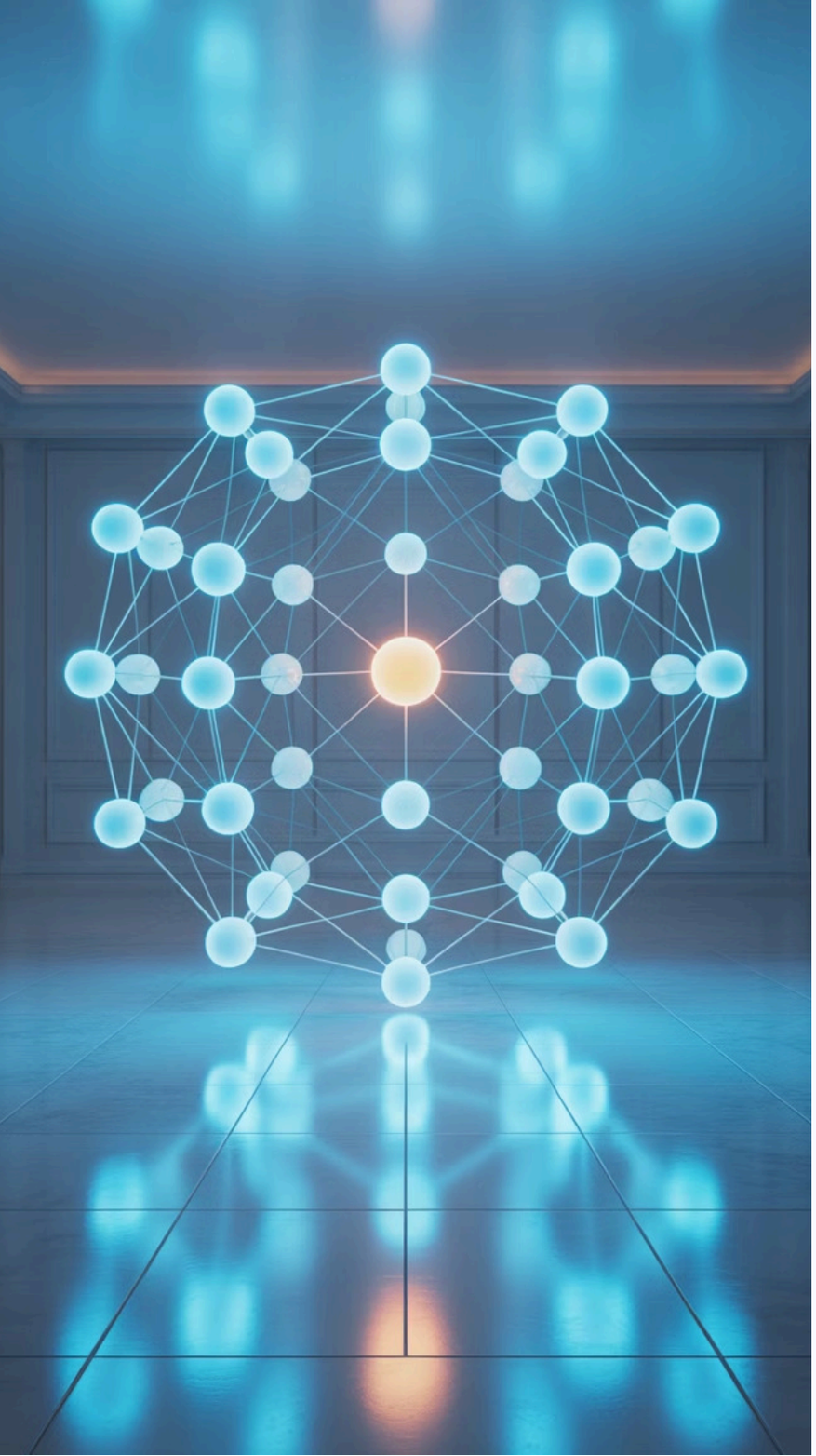
Key Principle 🔑

Propositional logic treats sentences as **abstract symbols** representing facts, not their meanings or content

Example Mapping:

- **P** = "It is raining"
- **Q** = "The ground is wet"
- **P → Q** = "If it's raining, then the ground is wet"

Logic focuses on relations between truths, not content itself



🌍 Semantics: Assigning Meaning

While syntax defines structure, **semantics** breathes life into logical expressions by defining their meaning:

Truth Conditions

Semantics specify **when a sentence is true or false** under different circumstances

Models

Every **model (m)** represents a possible world by assigning **True** or **False** to each atomic proposition

Interpretation

An **interpretation function** maps each sentence to its truth value in a given model.

🧮 Interpretation Function

The interpretation function $\mathbf{I}$ formally defines how truth values are computed recursively:

1

Base Case

$$I(P) \in \{True, False\}$$

Each atomic proposition is assigned either True or False

2

Negation Rule

$$I(\neg\alpha) = True \text{ iff } I(\alpha) = False$$

Negation flips the truth value of its operand

3

Conjunction Rule

$$I(\alpha \wedge \beta) = True \text{ iff } I(\alpha) = True \text{ and } I(\beta) = True$$

Both conjuncts must be true for conjunction to be true

4

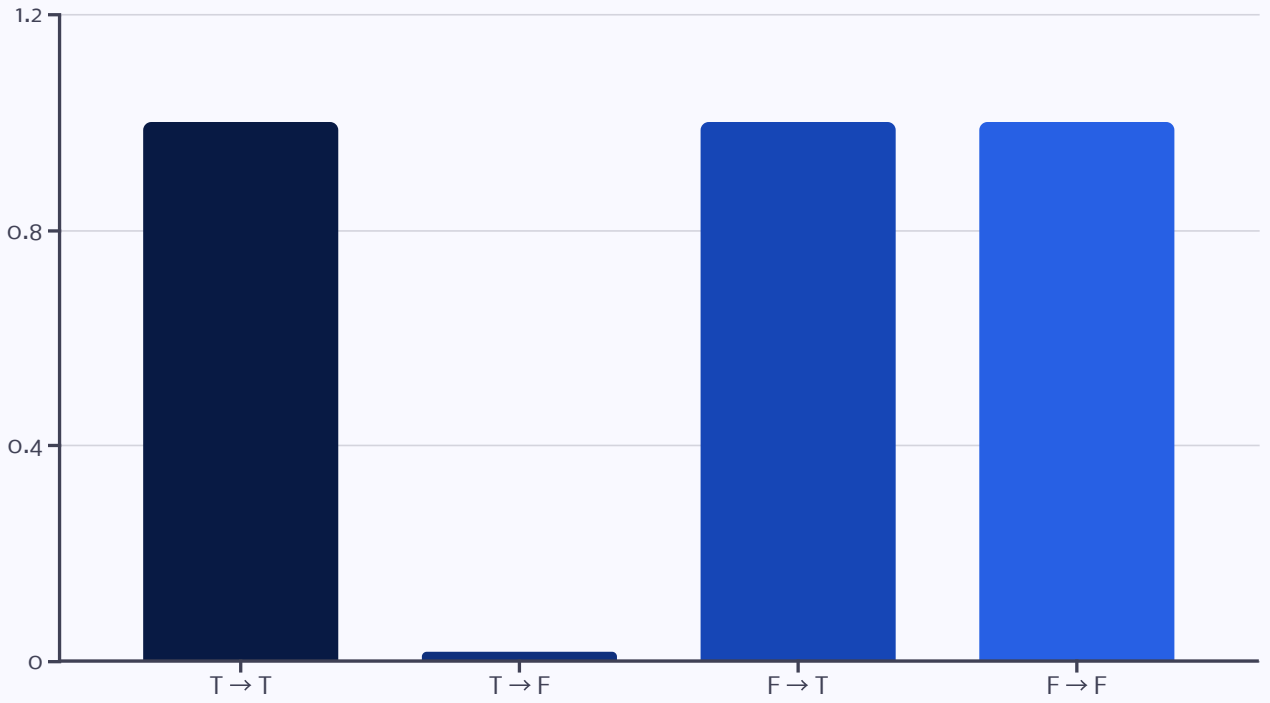
Disjunction Rule

$$I(\alpha \vee \beta) = True \text{ iff } I(\alpha) = True \text{ or } I(\beta) = True$$

At least one disjunct must be true

📊 Truth Table for Implication

The **implication operator (→)** is fundamental to logical reasoning, but its truth conditions can be counterintuitive:



Key Insight 💡

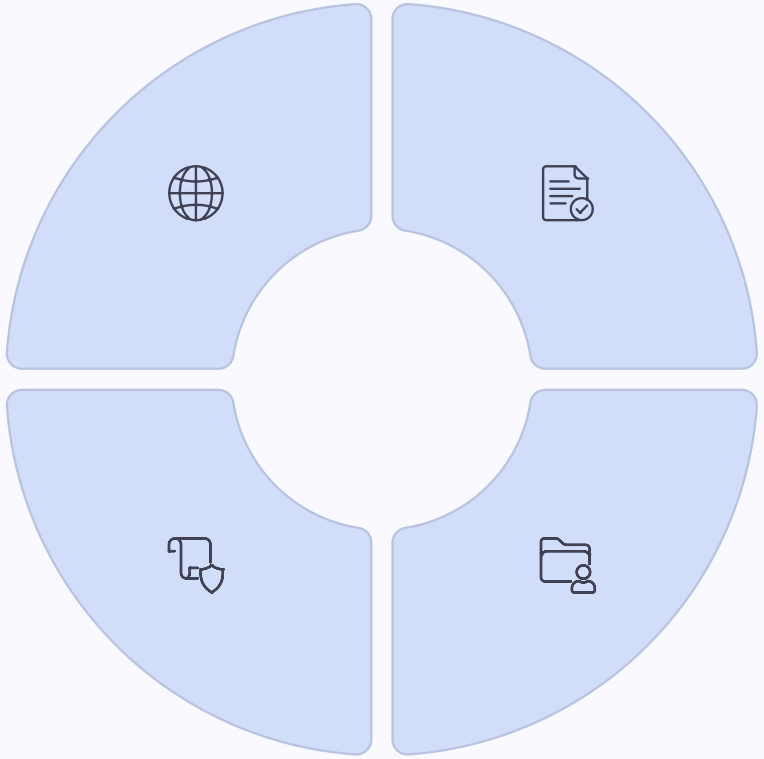
Implication is **only false** when the antecedent is **True** and the consequent is **False**

A false antecedent makes the implication vacuously true

🔗 Models and Possible Worlds

Model = World

Each **model** represents one complete possible world with specific truth assignments



Truth in Model

A sentence is **true in model m** if its truth assignment matches that world's configuration

Model Space

The set **M(KB)** contains all models where KB is true

KB Truth

A **knowledge base is true in m** if all sentences in KB are true in m

📘 Formal Definition of Entailment

The **entailment relation** is the cornerstone of logical inference in AI systems:

$$KB \models \alpha \iff M(KB) \subseteq M(\alpha)$$

1

Interpretation

KB entails α if every model satisfying KB also satisfies α

2

Logical Consequence

α is a necessary consequence of the knowledge in KB

3

Cannot Be False

There is **no possible world** where KB is true but α is false

🔍 Classic Example: Modus Ponens

KB = {P, (P → Q)} ⊢ Q

In model-theoretic terms: every world where both P and (P → Q) are true must also make Q true

🧩 Entailment vs. Inference

Two fundamental concepts in logic that are related but distinct:

Concept	Based On	Definition
Entailment (□)	Semantics	Truth in all models: α follows necessarily from KB in every possible world
Inference (□)	Syntax	Derivable via rules: α can be produced from KB using proof procedures

Entailment is about **what is true**, while inference is about **what we can prove** 🎯

🧠 Soundness and Completeness

These properties connect the semantic and syntactic views of logic:

Soundness ✓

If KB ⊢ α, then KB ⊨ α

Everything we can prove syntactically is semantically true

Guarantees: **No false proofs** – inference never produces incorrect results

Completeness ✓

If KB ⊨ α, then KB ⊢ α

Everything that is semantically true can be proven syntactically

Guarantees: **Can prove all truths** – inference is powerful enough to derive all valid conclusions

🎓 An ideal logical system is both **sound** and complete

⚙️ Monotonicity Property

$$\text{If } KB \models \alpha, \text{ then } (KB \wedge \beta) \models \alpha$$

What It Means 📖

Adding new knowledge to KB never invalidates previously derived truths

Once something is entailed, it remains entailed regardless of what else we learn

Implications 🔍

- **Knowledge accumulation** is safe and stable
- Cannot "undo" valid conclusions by learning more
- Critical for **incremental reasoning** in AI systems

Note: This differs from **non-monotonic reasoning**, where new information can overturn previous conclusions 🔄



🧩 Model Checking as Proof

Enumerate Models

Generate all possible truth assignments consistent with KB

Filter Valid Models

Keep only models where **all KB sentences** are true

Check Entailment

If α is true in all remaining models, then KB ⊨ α

⚠️ Computational Reality

Feasible for small knowledge bases with few propositions

Exponentially expensive for large KBs: with n propositions, there are 2n possible models

This motivates more efficient proof procedures like resolution

🔍 Model Checking Example: Step by Step

Let's demonstrate model checking with a concrete example:

KB = {P → Q, P} and we want to check if **KB □ Q**

T	T	T	T	YES	T
T	F	F	T	NO	Skip
F	T	T	F	NO	Skip
F	F	T	F	NO	Skip

Conclusion: Only Model 1 satisfies KB, and Q is true in Model 1, therefore KB □ Q ✅

Since Q is true in ALL models where KB is true, the entailment holds!

⚡ Model Checking Algorithm

The model checking algorithm provides a systematic way to determine entailment:

$$M(KB) = \{m | KB \text{ is true in model } m\}$$

01

Enumerate All Models

Generate all 2^n possible truth assignments for n propositions

02

Filter KB Models

Keep only models where every sentence in KB evaluates to True

03

Check Entailment

Verify if α is True in all remaining models

04

Return Result

If yes: KB ⊨ α, If no: KB ⊭ α

Time Complexity: O(2^n) where n = number of propositions

🎯 Model Checking: Practical Considerations

Advantages ✅

- **Guaranteed Correctness:** Always gives the right answer (sound and complete)
- **Conceptual Clarity:** Provides intuitive understanding of entailment
- **Universal Method:** Works for any propositional logic problem

Limitations ⚠️

- **Exponential Complexity:** 2^n models to check becomes intractable quickly
- **Memory Requirements:** Must store and evaluate many models simultaneously
- **Scalability Issues:** Impractical for real-world knowledge bases with many propositions

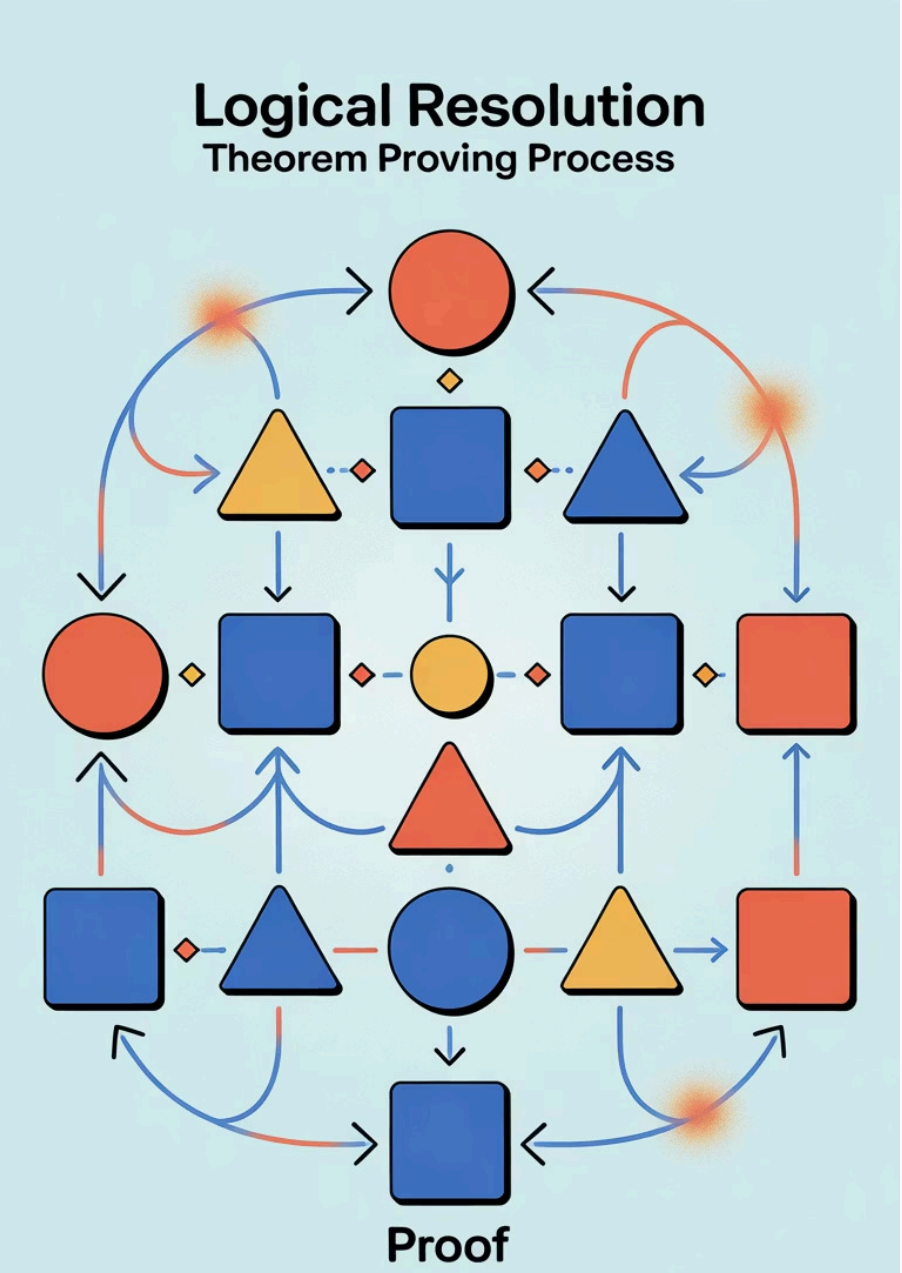
This motivates the need for more efficient proof procedures like **resolution** and modern **SAT solvers** that can handle larger problems.

⚖️ Deduction Rules Overview

Syntactic inference rules allow us to derive new sentences without enumerating models:

Rule	Formula	Example
Modus Ponens	(P → Q), P Q	If raining implies wet, and it's raining, then it's wet
Modus Tollens	(P → Q), ¬Q ¬P	If not wet, then not raining (contrapositive)
And-Elimination	(P ∧ Q) P	Extract one conjunct from conjunction
And-Introduction	P, Q (P Q)	Combine separate facts into conjunction
Or-Introduction	P (P Q)	Add disjunction to known fact

These rules are sound: they preserve truth through derivation 🎯



🧩 Resolution Rule

The **resolution rule** is the foundation of automated theorem proving:

$$(P \vee Q), (\neg P \vee R) \vdash (Q \vee R)$$

Combine Clauses

Take two clauses containing complementary literals (P and ¬P)

Remove Pair

Eliminate the complementary literals from both clauses

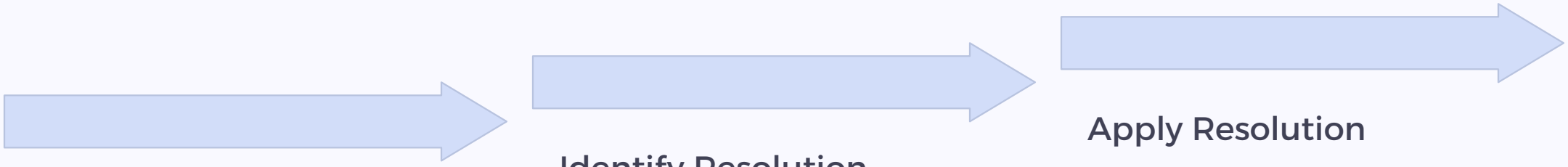
Form Resolvent

The **resolvent** contains all remaining literals from both clauses

🚀 Resolution is **refutation-complete**: can prove any valid entailment through contradiction

🔍 Resolution Example

Step-by-step application of the resolution rule:



Given Clauses

Clause 1: $(P \vee Q)$

Clause 2: $(\neg P \vee R)$

Notice P appears positive in clause 1, negative in clause 2

Identify Resolution

The literals P and $\neg P$ are complementary

These can be resolved together

Apply Resolution

Remove P and ¬P, combine remaining literals

Resolvent: $(Q \vee R)$ ✅

This is a **new derived clause** that follows logically

🧮 CNF and Clause Form

Why CNF Matters 🎯

Conjunctive Normal Form is required for applying the resolution algorithm effectively

Conversion Steps:

1. Eliminate ↔ and → using equivalences
2. Move ¬ inward using De Morgan's laws
3. Distribute ∨ over ∧ to get clause form
4. Result: conjunction of disjunctive clauses

Example Transformation 🔄

Original:

$$(P \rightarrow Q)$$

Step 1 - Eliminate implication:

$$(\neg P \vee Q)$$

Already in CNF!

More Complex:

$$(P \rightarrow Q) \wedge (Q \rightarrow R)$$

Becomes:

$$(\neg P \vee Q) \wedge (\neg Q \vee R)$$

🧠 Propositional Logic in AI Systems

Propositional logic provides the foundation for intelligent behavior in various AI applications:

Expert Systems

Encode domain expertise as logical rules for automated decision-making

SAT Solvers

Determine satisfiability of logical formulas, used in **verification** and **planning**

Safety Verification

Prove that systems meet **safety properties** before deployment

Agents can: **infer new facts**, **verify safety**, and **detect inconsistencies** 🤖



🎯 Wumpus Agent: Formal Reasoning

Applying propositional logic to the **Wumpus World** demonstrates practical AI reasoning:

01

Percept

Agent observes: **¬Breeze(1,1)** – no breeze at starting location

02

World Rule

Breeze(x,y) → (Pit(x+1,y) ∨ Pit(x−1,y) ∨ Pit(x,y+1) ∨ Pit(x,y−1))

Breeze indicates adjacent pit

03

Logical Inference

By **Modus Tollens**: ¬Breeze(1,1) → ¬Pit(2,1) ∧ ¬Pit(1,2)

04

Safe Action

Agent can **safely move** to (2,1) or (1,2) with logical certainty 🎯

⚙️ Computational Considerations

1

The Challenge 🚨

Model checking has exponential complexity: $O(2^n)$ where n = number of propositions

2

Efficient Algorithms 🚀

Need sophisticated inference algorithms like DPLL, **WalkSAT**, and modern **SAT solvers**

3

Expressiveness Limits 📊

Propositional logic struggles with **objects**, **relations**, and **generalization**

4

The Path Forward ➡️

These limitations motivate **First-Order Logic** for richer knowledge representation

🏁 Summary: Formal Reasoning Foundations

4	2	1
Core Pillars	**Key Properties**	**Foundation**
Syntax, Semantics, Entailment, Inference	Soundness and Completeness	Base layer of all logical reasoning

---

What You've Learned 🎓

- How to **formally represent knowledge** using propositional logic
- How to **prove what an agent knows** using semantic and syntactic methods
- The connection between formal reasoning and intelligent behavior

Coming Next ➡️

First-Order Logic (FOL) – extending propositional logic with **objects**, **relations**, **functions**, and **quantifiers** for more expressive knowledge representation 🚀